TB075/86.53 Procesamiento del Habla

Guía de ejercicios 2C 2026

Facultad de Ingeniería, Universidad de Buenos Aires
Dr. Kevin Michalewicz

Este documento concentra las consignas de las doce clases prácticas (P1 a P12). Está pensado como material de pizarrón y también para realizar en el hogar en caso de que algún ejercicio no se llegue a completar en clase. La parte de implementación en Python se encuentra en los notebooks asociados, referenciados al final de cada práctica.

La notación coincide con la vista en clase. Por ejemplo: $x_c(t)$ señal continua; $x[n]$ señal discreta; $\mathbf{y}_t$ vector de features para el frame t; $\lambda = (A, B, \boldsymbol{\pi})$ parámetros de un Modelo Oculto de Markov; $\alpha_t(i)$, $\beta_t(i)$, $\delta_t(i)$ variables de forward, backward y Viterbi; $\gamma_t(i)$ y $\xi_t(i, j)$ probabilidades *a posteriori* de estado y transición.

Índice

P1: Repaso de Procesamiento Digital de Señales **2**

P2: Análisis fonético **3**

P3: Coeficientes Cepstrales en las Frecuencias de Mel **4**

P4: Modelo de Mezcla Gaussiana **5**

P5: Modelos Ocultos de Markov **6**

P6: Reconocedor de dígitos con GMM-HMM **7**

P7: Modelos de lenguaje n-grama **8**

P8: Redes neuronales para habla **9**

P9: CNN y BiLSTM para palabras clave **10**

P10: CTC y alineamiento **11**

P11: ASR con modelos fundacionales **12**

P12: Síntesis de voz y evaluación **13**

P1: Repaso de Procesamiento Digital de Señales

Objetivos. Recuperar fluidez en la representación de una señal en tiempo y frecuencia. Estudiar los parámetros básicos de la Transformada de Fourier de Tiempo Corto (STFT). Distinguir frecuencia fundamental matemática de *pitch* percibido.

Ejercicio 1. Discriminación espectral de dos senoides

Sea la señal continua $x_c(t) = A\cos(2\pi f_1 t) + B\cos(2\pi f_2 t)$, con $f_1 = 100$ Hz, $f_2 = 200$ Hz, $A = B = 1$, muestreada a $f_s = 500$ Hz para obtener $x[n] = x_c(nT_s)$.

(a) Calcular analíticamente el espectro $X_c(\Omega)$ de $x_c(t)$ y el espectro $X(\omega)$ de $x[n]$ para $f_s > 2f_{\max} = 400$ Hz.

(b) Tomando una ventana temporal $x_t[n]$ de duración L muestras, determinar el menor L y el menor N_{FFT} que hacen evidente la presencia de las dos senoidales en el espectro discreto. Justificar usando la respuesta en frecuencia de la ventana rectangular.

Ejercicio 2. Pitch matemático vs. pitch percibido

Para una señal $x_c(t) = A\cos(2\pi f_1 t) + B\cos(2\pi f_2 t)$, determinar el período fundamental T_0 y la frecuencia fundamental $F_0 = 1/T_0$ en cada caso.

(a) $f_1 = 220$ Hz, $f_2 = 440$ Hz, $A = B = 1$.

(b) $f_1 = 220$ Hz, $f_2 = 660$ Hz, $A = 2, B = 1$.

(c) $f_1 = 660$ Hz, $f_2 = 880$ Hz, $A = 1, B = 0{,}1$.

¿Qué papel juegan A y B en la definición de F_0 matemáticamente? ¿Y perceptualmente? Anticipar qué se va a escuchar antes de sintetizar.

Notebook

Acceder al archvio `P1_Intro.ipynb`. En el ejercicio 3 comparamos los resultados de los ejercicios 1 y 2 con sus versiones obtenidas computacionalmente. Luego se proponen los ejercicios 4 y 5 que cubren el cómputo de espectrogramas y DTMF.

P2: Análisis fonético

Objetivo. Familiarizarse con la lectura de espectrogramas de habla real, identificar fonemas y formantes, e interpretar la prosodia (F_0).

Ejercicio 1. Predicción de formantes

Para las cinco vocales del español $/a/, /e/, /i/, /o/, /u/$:

(a) Indicar la posición articulatoria (anterior, central, posterior) y la abertura (cerrada, media, abierta).

(b) Predecir el orden cualitativo de F_1 y F_2. Justificar.

(c) Dibujar a mano alzada el diagrama (F_1, F_2) de las cinco vocales.

Ejercicio 2. Influencia del hablante

El tracto vocal femenino es aproximadamente un 15 % más corto que el masculino (estadísticamente hablando). Predecir el efecto sobre F_1, F_2, F_3 y sobre F_0. ¿Cómo afecta esto al diseño de un sistema de ASR si entrena solamente con voces masculinas?

Notebook

Acceder al archvio `P2_Fonetica.ipynb`. En el ejercicio 3 sintetizamos las cinco vocales por modelo fuente-filtro, en el ejercicio 4 verificamos lo obtenido en el ejercicio 1, y en el ejercicio 5 interpretamos el espectrograma de una frase corta.

P3: Coeficientes Cepstrales en las Frecuencias de Mel

Objetivo. Entender el pipeline MFCC como una representación compacta de la envolvente espectral. Diseñar un banco mel simple, interpretar el rol del logaritmo y de la DCT, e implementar MFCC en Python sobre señales de habla sintética.

Ejercicio 1. Banco de filtros en la escala mel

Queremos construir un banco de $M = 5$ filtros triangulares entre $f_{\text{mín}} = 100$ Hz y $f_{\text{máx}} = 4000$ Hz. Usamos

$$\text{mel}(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right), \qquad f(\text{mel}) = 700\left(10^{\text{mel}/2595} - 1\right).$$

(a) Convertir 100 Hz y 4000 Hz a escala mel.

(b) Para construir 5 filtros triangulares, explicar por qué es necesario considerar $M+2 = 7$ frecuencias.

(c) Distribuir esos 7 puntos equiespaciados en escala mel y convertirlos a Hz. Identificar las 5 frecuencias centrales.

(d) Esbozar cualitativamente los filtros. ¿Por qué se ensanchan en Hz al aumentar la frecuencia?

Ejercicio 2. Del espectro al vector MFCC

Considerar el pipeline de MFCC para un frame $x_t[n]$:

$$x_t[n] \longrightarrow S[k,t] \longrightarrow E_m[t] \longrightarrow \widetilde{E}_m[t] \longrightarrow c_d[t].$$

(a) Indicar qué representa cada cantidad: $S[k,t]$, $E_m[t]$, $\widetilde{E}_m[t]$ y $c_d[t]$.

(b) Explicar por qué se toma logaritmo antes de la DCT. Relacionar con compresión de rango dinámico y con el modelo fuente-filtro.

(c) Explicar por qué la DCT es útil al considerar GMM con covarianza diagonal.

(d) Si se conservan $C = 13$ coeficientes estáticos y se agregan deltas y delta-deltas, ¿cuál es la dimensión final D del vector $\mathbf{y}_t$?

Notebook

Acceder al archivo `P3_MFCC.ipynb`. En el ejercicio 3 implementamos el pipeline MFCC desde cero para una señal de habla sintética. En el ejercicio 4 verificamos empíricamente que la DCT reduce correlaciones entre dimensiones y permite justificar el uso de covarianzas diagonales en GMM.

P4: Modelo de Mezcla Gaussiana

Objetivos. Aplicar la regla MAP con modelos gaussianos, realizar un paso del algoritmo EM para una mezcla gaussiana e interpretar estas densidades como modelos de emisión para Modelos Ocultos de Markov.

Ejercicio 1. Verosimilitud gaussiana de una vocal

Sea $\mathbf{y} = [500\ 1500]^\top$ Hz un punto del plano (F_1, F_2). Evaluar su log-verosimilitud bajo una clase modelada por

$$\boldsymbol{\mu} = \begin{bmatrix} 300 \\ 2200 \end{bmatrix}, \qquad \boldsymbol{\Sigma} = \begin{bmatrix} 1600 & 0 \\ 0 & 10000 \end{bmatrix}.$$

(a) Calcular la distancia de Mahalanobis al cuadrado

$$\delta^2 = (\mathbf{y} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{y} - \boldsymbol{\mu}).$$

(b) Calcular $\log p(\mathbf{y})$ usando la densidad gaussiana en $\mathbb{R}^2$.

(c) Interpretar el resultado: ¿es razonable asignar $\mathbf{y}$ a esa clase? ¿Qué vocal del español podría corresponder a la media $\boldsymbol{\mu}$?

Ejercicio 2. Un paso de EM

Sea un dataset unidimensional con tres observaciones:

$$y^{(1)} = 1, \qquad y^{(2)} = 2, \qquad y^{(3)} = 5.$$

Se modela con una GMM de dos componentes, con parámetros iniciales

$$\omega_1 = \omega_2 = 0{,}5, \qquad \mu_1 = 1{,}5, \qquad \mu_2 = 4{,}5, \qquad \sigma_1^2 = \sigma_2^2 = 1.$$

Para que el cálculo analítico sea razonable, mantener las varianzas fijas en 1 y hacer un paso de EM solo para pesos y medias.

(a) Paso E: calcular las responsabilidades

$$\gamma_{im} = P(Z^{(i)} = m | Y^{(i)} = y^{(i)}).$$

(b) Paso M: reestimar ω_m y μ_m.

(c) Interpretar: ¿qué componente explica principalmente a cada punto? ¿Por qué el punto $y = 2$ no queda asignado con responsabilidad exactamente 1 a la primera componente?

Notebook

Acceder al archivo `P4_GMM.ipynb`. En el ejercicio 3 verificamos numéricamente el cálculo de los ejercicios 1 y 2 y comprobamos que un paso de EM aumenta la log-verosimilitud. En el ejercicio 4 entrenamos un clasificador generativo de vocales en el plano (F_1, F_2) usando gaussianas/GMM y visualizamos sus regiones de decisión.

P5: Modelos Ocultos de Markov

Objetivo. Calcular las cantidades básicas de un Modelo Oculto de Markov discreto: forward, backward, probabilidades *a posteriori* de estado y camino de Viterbi.

Tomemos un HMM de $N = 2$ estados y dos símbolos de observación $\mathcal{V} = \{\mathtt{x}, \mathtt{z}\}$. Los parámetros son

$$\boldsymbol{\pi} = \begin{bmatrix} 0{,}6 \\ 0{,}4 \end{bmatrix}, \qquad A = \begin{bmatrix} 0{,}7 & 0{,}3 \\ 0{,}4 & 0{,}6 \end{bmatrix}, \qquad B = \begin{bmatrix} 0{,}9 & 0{,}1 \\ 0{,}2 & 0{,}8 \end{bmatrix}.$$

La fila j de B da las probabilidades de emisión desde el estado j, mientras que la primera columna corresponde a `x` y la segunda a `z`. La secuencia observada es

$$y_{1:3} = (\mathtt{x}, \mathtt{z}, \mathtt{x}).$$

Ejercicio 1. Forward y backward

(a) Calcular $\alpha_t(i)$ para $t = 1, 2, 3$ e $i = 1, 2$.

(b) Calcular $P(y_{1:3}|\lambda) = \sum_i \alpha_3(i)$. ¿Por qué usamos como notación P en vez de p?

(c) Calcular $\beta_t(i)$ para $t = 3, 2, 1$.

(d) Verificar que

$$\sum_i \alpha_t(i)\beta_t(i) = P(y_{1:3}|\lambda)$$

para $t = 1, 2, 3$.

Ejercicio 2. Decodificación por probabilidad *a posteriori* y por Viterbi

(a) Calcular

$$\gamma_t(i) = P(Q_t = i | Y_{1:3} = y_{1:3}, \lambda) = \frac{\alpha_t(i)\beta_t(i)}{P(y_{1:3}|\lambda)}.$$

Verificar que $\sum_i \gamma_t(i) = 1$ para cada t.

(b) Decodificar por probabilidad *a posteriori* local:

$$\widehat{q}_t^{\,\text{local}} = \arg\max_i \gamma_t(i).$$

(c) Calcular $\delta_t(i)$ y $\psi_t(i)$ (Viterbi).

(d) Determinar el camino más probable $q^*_{1:3}$ y computar

$$P^* = \max_{q_{1:3}} P(Q_{1:3} = q_{1:3}, Y_{1:3} = y_{1:3}|\lambda).$$

(e) ¿Coinciden los resultados de los ítemos (b) y (d) en este caso? ¿Y en general? ¿Qué tan robusto es cada uno a las transiciones prohibidas?

Notebook

Acceder al archivo `P5_HMM.ipynb`. En este ejercicio único replicamos los cálculos, implementamos las recursiones de forma general y visualizamos las cantidades sobre el trellis.

P6: Reconocedor de dígitos con GMM-HMM

Objetivo. Entender cómo un reconocedor de palabras aisladas usa un HMM por palabra: cada modelo asigna una verosimilitud a la secuencia acústica y se elige el dígito con mayor score. Trabajamos con un subconjunto del *Free Spoken Digit Dataset*, formado por audios reales de dígitos aislados en inglés.

Ejercicio 1. Conteo de parámetros

Un reconocedor de palabras aisladas tiene un vocabulario $\mathcal{W} = \{\text{"cero"}, \text{"uno"}, \ldots, \text{"nueve"}\}$, por lo que $|\mathcal{W}| = 10$. Cada palabra se modela con un HMM *left-to-right* de $N = 5$ estados. Cada estado emite vectores MFCC de dimensión $D = 13$ mediante una mezcla de $M = 4$ gaussianas con covarianza diagonal.

(a) Calcular cuántos parámetros libres de emisión tiene un estado, es decir, pesos de mezcla, medias y varianzas.

(b) Calcular cuántos parámetros libres de emisión tiene una palabra completa.

(c) Suponiendo que la topología permite solamente *self-loops* y avances al estado siguiente, que la distribución inicial está fija en el primer estado y que el último estado es absorbente, calcular cuántos parámetros de transición libres hay por palabra.

(d) Calcular el número total de parámetros libres para las diez palabras.

Ejercicio 2. Topología y duración

Consideramos caminos que comienzan en el primer estado y deben alcanzar el último estado. Si todos los estados deben visitarse al menos una vez, un HMM *left-to-right* de N estados requiere como mínimo N frames. El desplazamiento entre frames es de 10 ms.

(a) Si usamos $N = 5$, ¿cuál es la duración mínima aproximada de una palabra?

(b) Para palabras de aproximadamente 400 ms y 250 ms, ¿esa duración mínima resulta restrictiva o permisiva?

(c) En un sistema más realista, ¿conviene utilizar el mismo N para todas las palabras? Relacionar con la regla práctica de tres estados por fonema.

Notebook

Acceder al archivo `P6_GMM_HMM.ipynb`. Cargamos audios reales del *Free Spoken Digit Dataset*, extraemos los MFCC y estimamos un HMM gaussiano *left-to-right* por dígito mediante una segmentación uniforme. Esta estimación se realiza en una única pasada y sin aplicar Baum-Welch. Luego clasificamos por máxima verosimilitud y obtenemos un alineamiento final mediante Viterbi.

P7: Modelos de lenguaje n-grama

Objetivos. Estimar un modelo bigrama sobre un corpus pequeño, observar el problema de los ceros, aplicar suavizado add-one y calcular perplejidad.

Ejercicio 1. Del corpus al modelo bigrama

Usar el corpus provisto en el notebook.

(a) Calcular

$$P_{\text{MV}}(\texttt{"pescado"}|\texttt{"come"}), \qquad P_{\text{MV}}(\texttt{"perro"}|\texttt{"gato"}).$$

(b) Calcular con add-one

$$P_{+1}(\texttt{"pescado"}|\texttt{"come"}), \qquad P_{+1}(\texttt{"perro"}|\texttt{"gato"}).$$

Explicar qué ocurre con la masa asignada a eventos vistos y no vistos.

(c) Calcular la probabilidad y la perplejidad de

$$\texttt{"<s> el gato come carne </s>"}$$

usando add-one. Tomar como longitud las cinco transiciones bigrama.

Ejercicio 2. Combinación acústica y lingüística

Después de reconocer el prefijo `"el"`, el decoder considera dos posibles continuaciones. Para los frames correspondientes a la próxima palabra, los incrementos de score acústico y las probabilidades bigrama son

w	$\Delta s_{\text{ac}}(w)$	$P(w\|\texttt{"el"})$
`"gato"`	$-10{,}5$	0,6
`"perro"`	$-10{,}3$	0,4

El incremento del score del decoder al agregar la palabra w es

$$\Delta s_{\text{dec}}(w) = \Delta s_{\text{ac}}(w) + \eta \log P(w|\texttt{"el"}) + \rho,$$

donde η es el peso del modelo de lenguaje y ρ es el término de inserción correspondiente a la nueva palabra. Usar logaritmo natural.

(a) ¿Qué palabra se elegiría considerando solamente el score acústico?

(b) Para $\eta = 1$, calcular $\Delta s_{\text{dec}}(w)$ para ambas continuaciones, dejando indicado el término ρ, y dar la decisión final.

(c) Hallar el valor crítico η_c para el cual ambas continuaciones obtienen el mismo score. Indicar para qué valores de η se elige `"gato"`.

(d) El score acumulado de una hipótesis completa contiene el término $K\rho$, donde K es su cantidad de palabras. Explicar por qué este término no modifica la comparación entre las dos continuaciones.

Notebook

Acceder al archivo `P7_LM_ngrama.ipynb`. Completamos las funciones de conteo, probabilidad y perplejidad. Compararamos distintos valores de α e implementamos la generación por muestreo.

P8: Redes neuronales para habla

Objetivos. Calcular gradientes para una red neuronal de una capa oculta, entrenar un MLP sobre audios reales y distinguir clasificación de palabras y emisiones de un DNN-HMM.

Ejercicio 1. Cálculo de gradientes para una red de una capa oculta

Consideremos la entrada $\mathbf{u} = [1, -1]^\top$ y una red neuronal que consta de una capa oculta con dos neuronas (con ReLU), y una capa de salida de tres neuronas con activación softmax para resolver un problema de clasificación multiclase:

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{u} + \mathbf{b}^{(1)}, \quad \mathbf{h}^{(1)} = \mathrm{ReLU}(\mathbf{z}^{(1)}), \quad \mathbf{z}^{(2)} = W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}, \quad \widehat{\mathbf{p}} = \mathrm{softmax}(\mathbf{z}^{(2)}).$$

Los parámetros (matrices de pesos y vectores de *bias*) son:

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

La clase correcta es s_2 (asumiendo índices 1, 2 y 3), así que $\mathcal{L} = -\log \widehat{p}_2$.

(a) Calcular preactivaciones, activaciones, logits, probabilidades y la *loss*/pérdida.

(b) Calcular $\boldsymbol{\delta}^{(2)}$, $\partial\mathcal{L}/\partial W^{(2)}$, $\partial\mathcal{L}/\partial \mathbf{b}^{(2)}$, $\boldsymbol{\delta}^{(1)}$, $\partial\mathcal{L}/\partial W^{(1)}$ y $\partial\mathcal{L}/\partial \mathbf{b}^{(1)}$. Recordar que $\boldsymbol{\delta}^{(2)} = \widehat{\mathbf{p}} - \mathbf{e}_2$ y $\boldsymbol{\delta}^{(1)} = (W^{(2)})^T\boldsymbol{\delta}^{(2)} \odot \mathbb{I}_{\{\mathbf{z}^{(1)}>0\}}$.

Ejercicio 2. De probabilidad *a posteriori* a score acústico

Una red de un sistema DNN-HMM estima, para un frame $\mathbf{y}_t$, probabilidades *a posteriori* 0,60, 0,25 y 0,15 para los estados 1, 2 y 3, respectivamente. Sus priors son 0,75, 0,15 y 0,10, respectivamente.

(a) Usar Bayes para obtener cantidades proporcionales a $b_j(\mathbf{y}_t) = p(\mathbf{y}_t|Q_t = j)$ y sus logaritmos, omitiendo el término común a todos los estados.

(b) ¿Difiere el estado seleccionado mediante probabilidades *a posteriori* de aquel que se elegiría basándose en el mayor score acústico? ¿Alcanza esta comparación para decir algo sobre el camino de Viterbi del HMM?

Notebook

Acceder al archivo `P8_Redes_Neuronales.ipynb`. Entrenamos un MLP con 3000 audios reales de dígitos en inglés de *Free Spoken Digit Dataset*.

P9: CNN y BiLSTM para palabras clave

Objetivo. Entender cómo una CNN extrae patrones locales de una representación log-mel y cómo un BiLSTM incorpora contexto temporal para clasificar palabras aisladas.

Ejercicio 1. Preparación de secuencias para LSTM

Una señal de un segundo se muestrea a $f_s = 16000$ Hz. Se usa la ventana de Hamming de $L = 400$ muestras, hop de 160 muestras, $N_{\text{FFT}} = 400$ y $M = 40$ filtros de mel. Sólo se conservan frames completos. La CNN recibe un canal y aplica:

- Conv2D con 8 filtros de 3×3, stride 1, padding 1 y un *bias* por filtro;
- ReLU y MaxPool2D de 2×1, con stride 2×1.

El primer eje espacial corresponde a la frecuencia y el segundo al tiempo.

(a) Calcular el número de frames T. Obtener las dimensiones de la entrada, de la salida de la convolución y de la salida del pooling (no incluir el eje correspondiente al *batch*).

(b) Redefinimos la salida como una secuencia temporal. Indicar la longitud de la secuencia y la dimensión D de las features de entrada para la red LSTM.

(c) Contar los parámetros entrenables de la convolución. ¿ReLU y MaxPool agregan parámetros?

Ejercicio 2. BiLSTM: parámetros, resumen y causalidad

Procesamos la secuencia anterior mediante una capa BiLSTM con $H = 32$ unidades ocultas (por dirección). En `nn.LSTM`, cada dirección almacena matrices de pesos con dimensiones $4H \times D$ y $4H \times H$, y *dos* vectores de *bias* de dimensión $4H$. Luego, promediamos sus salidas temporales y aplicamos una capa lineal con *bias* para clasificar cuatro palabras.

(a) Contar los parámetros entrenables de una dirección y del BiLSTM completo.

(b) Determinar la dimensión del vector de salida del BiLSTM en cada paso de tiempo, así como la del vector resultante tras el promedio temporal. Calcular la cantidad de parámetros de la capa lineal y el total acumulado de toda la arquitectura (incluyendo la CNN previa).

(c) Explicar por qué promediar temporalmente las salidas del BiLSTM no elimina la sensibilidad al orden secuencial, a diferencia de lo que ocurría al promediar los MFCC antes del MLP en la P8.

(d) ¿Por qué este modelo es adecuado para clasificación *offline*? Si quisiéramos procesar el audio en tiempo real (*streaming*), ¿alcanzaría únicamente con reemplazar el BiLSTM por un LSTM unidireccional para evitar mirar frames futuros?

Notebook

Acceder a `P9_CNN-BiLSTM.ipynb`. Las consignas de los ejercicios 3 y 4 están en el notebook. Clasificamos `"yes"`, `"no"`, `"up"` y `"down"` de *Mini Speech Commands*, con particiones por hablante. Es necesario descargar un archivo ZIP con los datos.